

CHAPTER 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1. Introduction

A well-designed database is essential for the smooth operation of an e-commerce platform. In an IKEA furniture e-commerce system, large amounts of data such as customer information, products, orders, payments, shipments, reviews, and inventory must be stored accurately and efficiently. A proper database design helps organize this information while reducing duplication and maintaining consistency.

Functional Dependency (FD) defines the relationship between attributes within a table. It states that the value of a primary key uniquely determines all other related attributes. Functional dependencies are important because they help identify how data should be organized and serve as the foundation for database normalization.

Redundancy refers to storing the same information multiple times. Excessive redundancy increases storage requirements and may lead to inconsistent data during updates. By separating related information into different tables and connecting them through foreign keys, redundancy is minimized and data integrity is maintained.

Normalization is the process of organizing data into well-structured tables to eliminate redundancy and prevent data anomalies. A normalized database improves consistency, simplifies maintenance, and supports future scalability.

Database designers also analyze three common data anomalies:

- Insert Anomaly – Difficulty inserting new data without additional unrelated information.
- Update Anomaly – The same information must be updated in multiple places.
- Delete Anomaly – Deleting one record accidentally removes important related data.

The IKEA Furniture E-Commerce Database has been designed using separate tables such as Customer, Product, Order, Payment, Shipment, Inventory, Review, Cart, Wishlist, Supplier, and Coupon. These tables are connected through primary and foreign keys, ensuring data consistency, reducing redundancy, and supporting efficient database operations.

This report analyzes the functional dependencies, redundancy, and data anomalies of each table and explains how the database design follows normalization principles to create a reliable and scalable furniture e-commerce system.

4.2. Normalization Overview

Normalization is the process of organizing data into well-structured tables to reduce redundancy and eliminate data anomalies. It ensures that each table stores only related information and that relationships between tables are maintained using primary and foreign keys.

The IKEA Furniture E-Commerce Database follows the first three normal forms.

Normal Form	Status	Explanation
First Normal Form (1NF)	Achieved	All attributes contain atomic values, and there are no repeating groups.
Second Normal Form (2NF)	Achieved	Every non-key attribute depends completely on the primary key. Partial dependencies have been eliminated.
Third Normal Form (3NF)	Achieved	Non-key attributes depend only on the primary key, and transitive dependencies have been removed by separating related information into different tables.

Because of normalization:

- Customer information is stored separately from orders.
- Product categories are maintained independently.
- Inventory is managed separately from products.
- Customer addresses are stored in a dedicated table.
- Shopping cart and wishlist information are separated from customer records.
- AI recommendations, notifications, analytics, and search history are stored independently to avoid unnecessary duplication.

This design improves data consistency, minimizes redundancy, and allows the database to scale efficiently as the number of customers, products, and transactions increases.

4.3. Table Analysis

4.3.1 Customer Table

Functional Dependency

The primary key of the Customer table is Customer_ID.

The functional dependency is:

Customer_ID → Customer_Name, Email, Phone_Number, Password, Registration_Date, Customer_Status

This dependency exists because each customer is assigned a unique Customer_ID. Once the Customer_ID is known, the database can uniquely identify all other customer details associated with that customer.

If the business enforces unique email addresses, then:

Email → Customer_ID

can also act as a candidate key.

Redundancy Analysis

- Separation of Data: Storing customer details separately from orders, addresses, carts, wishlists, reviews, and notifications prevents repeating profile data across multiple tables.
- Foreign Key Links: Related tables reference Customer_ID rather than duplicating customer details in every record.
- Normalization Impact: This design minimizes redundancy and adheres strictly to relational database normalization standards.

Data Anomalies

- Insert Anomaly: Prevented. Customers can register and create an account independently without having to place an order first.
- Update Anomaly: Prevented. Customer details exist in a single location, so updating contact info requires modifying only one record.
- Delete Anomaly: Prevented. Deleting or canceling order records does not remove or affect the customer's account profile.

Summary

Item	Description
Primary Key	Customer_ID
Functional Dependency	Customer_ID → Customer_Name, Email, Phone_Number, Password, Registration_Date, Customer_Status
Candidate Key	Email (if unique)
Redundancy	Very Low
Insert Anomaly	Prevented
Update Anomaly	Prevented

Item	Description
Delete Anomaly	Prevented

4.3.2 Customer_Address Table

Functional Dependency

Address_ID → Customer_ID, Address_Line, City, State, Pincode, Country, Address_Type

Each address is uniquely identified by Address_ID. Since one customer can have multiple addresses (Home, Office, etc.), Customer_ID is maintained as a foreign key.

Redundancy Analysis

The Customer_Address table stores customer addresses separately, allowing multiple addresses without duplicating customer information. This reduces redundancy and supports normalization.

Data Anomalies

- Insert Anomaly: Multiple addresses can be added for an existing customer without duplicating customer details.
- *Example:* A customer saves both Home and Office addresses.
- Update Anomaly: Updating an address requires changing only the corresponding address record.
- Delete Anomaly: Deleting one address does not remove the customer account or other saved addresses.

Summary

Item	Description
Primary Key	Address_ID
Functional Dependency	Address_ID → Customer_ID, Address_Line, City, State, Pincode, Country, Address_Type
Redundancy	Very Low
Anomalies	Minimized

4.3.3 Supplier Table

Functional Dependency

Supplier_ID → Supplier_Name, Email, Phone_Number

Each supplier is uniquely identified by Supplier_ID. If email is unique, it can also serve as a candidate key.

Redundancy Analysis

Supplier information is stored separately, while the Product table stores only Supplier_ID. This avoids repeating supplier details for every product.

Data Anomalies

- Insert Anomaly: A supplier can be added before any products are supplied.
- *Example:* IKEA registers a new furniture supplier.
- Update Anomaly: Supplier details are updated in one place only.
- *Example:* The supplier changes their contact number.
- Delete Anomaly: Deleting a product does not remove supplier information.

Summary

Item	Description
Primary Key	Supplier_ID
Functional Dependency	Supplier_ID → Supplier_Name, Email, Phone_Number
Candidate Key	Email (if unique)
Redundancy	Very Low
Anomalies	Minimized

4.3.4 Category Table

Functional Dependency

Category_ID → Category_Name, Description

Each category is uniquely identified by Category_ID.

Redundancy Analysis

Category details are stored separately, while products reference the category using Category_ID. This avoids repeating category information.

Data Anomalies

- Insert Anomaly: New categories can be added without creating products.
- *Example:* "Outdoor Furniture" is added before products are available.
- Update Anomaly: Changing a category name requires updating only one record.
- Delete Anomaly: Deleting a product does not remove its category.

Summary

Item	Description
Primary Key	Category_ID
Functional Dependency	Category_ID → Category_Name, Description
Redundancy	Very Low
Anomalies	Minimized

4.3.5 Product Table

Functional Dependency

Product_ID → Product_Name, Description, Price, Brand, Weight, Color, Material, Category_ID, Supplier_ID

Each product is uniquely identified by Product_ID.

Redundancy Analysis

The Product table stores only product-specific information. Category and supplier details are referenced using foreign keys, reducing duplicate data and maintaining consistency.

Data Anomalies

- Insert Anomaly: A product can be added once its category and supplier exist.
- *Example:* A new dining table is added using existing Category_ID and Supplier_ID.
- Update Anomaly: Product information is updated in one record, while supplier or category changes are managed in their respective tables.
- Delete Anomaly: Deleting a product does not remove its supplier or category information.

Summary

Item	Description
Primary Key	Product_ID
Functional Dependency	Product_ID → Product_Name, Description, Price, Brand, Weight, Color, Material, Category_ID, Supplier_ID
Redundancy	Very Low
Anomalies	Minimized

4.3.6 Warehouse Table

Functional Dependency

Warehouse_ID $\rightarrow$ Warehouse_Name, Location

Each warehouse has a unique Warehouse_ID that determines its name and location.

Redundancy Analysis

Warehouse information is stored separately so that multiple products can be linked to the same warehouse without repeating warehouse details.

Data Anomalies

- Insert Anomaly: A new warehouse can be added before storing any products.
- Update Anomaly: Changing the warehouse location requires updating only one record.
- Delete Anomaly: Removing a product does not delete the warehouse information.

Summary

Item	Description
Primary Key	Warehouse_ID
Functional Dependency	Warehouse_ID $\rightarrow$ Warehouse_Name, Location
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.7 Inventory Table

Functional Dependency

Inventory_ID $\rightarrow$ Product_ID, Warehouse_ID, Stock_Quantity

Each inventory record uniquely identifies the stock of a product available in a particular warehouse.

Redundancy Analysis

Inventory is maintained separately from the Product table. This avoids repeating stock information for every product and allows the same product to exist in multiple warehouses.

Data Anomalies

- Insert Anomaly: Inventory can be created after a product and warehouse are available.
- Update Anomaly: Updating stock quantity affects only one inventory record.

- Delete Anomaly: Removing inventory does not delete the product or warehouse information.

Summary

Item	Description
Primary Key	Inventory_ID
Functional Dependency	Inventory_ID $\rightarrow$ Product_ID, Warehouse_ID, Stock_Quantity
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.8 Cart Table

Functional Dependency

Cart_ID $\rightarrow$ Customer_ID, Created_Date

Each cart belongs to one customer and stores the cart creation date.

Redundancy Analysis

The Cart table stores only customer cart information, while products are maintained separately in the Cart_Item table. This avoids repeating cart details for every product.

Data Anomalies

- Insert Anomaly: A cart can be created before adding any products.
- Update Anomaly: Updating the cart date requires changing only one record.
- Delete Anomaly: Deleting a cart does not remove customer information.

Summary

Item	Description
Primary Key	Cart_ID
Functional Dependency	Cart_ID $\rightarrow$ Customer_ID, Created_Date
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.9 Cart_Item Table

Functional Dependency

$\text{Cart_Item_ID} \rightarrow \text{Cart_ID}, \text{Product_ID}, \text{Quantity}$

Each Cart_Item record represents one product added to a cart.

Redundancy Analysis

The Cart_Item table resolves the relationship between Cart and Product without duplicating product information.

Data Anomalies

- Insert Anomaly: Products can be added only after the cart exists.
- Update Anomaly: Changing the quantity updates only one record.
- Delete Anomaly: Removing a product from the cart does not delete the cart or product.

Summary

Item	Description
Primary Key	Cart_Item_ID
Functional Dependency	$\text{Cart_Item_ID} \rightarrow \text{Cart_ID}, \text{Product_ID}, \text{Quantity}$
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.10 Wishlist Table

Functional Dependency

$\text{Wishlist_ID} \rightarrow \text{Customer_ID}, \text{Created_Date}$

Each wishlist belongs to one customer.

Redundancy Analysis

Wishlist details are stored separately from customer information, allowing customers to maintain wishlists without duplicating their personal details.

Data Anomalies

- Insert Anomaly: A wishlist can be created before adding products.
- Update Anomaly: Updating wishlist information affects only one record.
- Delete Anomaly: Deleting a wishlist does not remove customer information.

Summary

Item	Description
Primary Key	Wishlist_ID
Functional Dependency	Wishlist_ID $\rightarrow$ Customer_ID, Created_Date
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.11 Wishlist_Item Table

Functional Dependency

Wishlist_Item_ID $\rightarrow$ Wishlist_ID, Product_ID

Each record represents one product saved in a customer's wishlist.

Redundancy Analysis

Instead of storing multiple product details inside the Wishlist table, products are maintained separately through foreign keys. This keeps the database normalized.

Data Anomalies

- Insert Anomaly: Products can be added only after the wishlist exists.
- Update Anomaly: Product changes are reflected automatically through the Product table.
- Delete Anomaly: Removing a product from the wishlist does not delete the product or the wishlist.

Summary

Item	Description
Primary Key	Wishlist_Item_ID
Functional Dependency	Wishlist_Item_ID $\rightarrow$ Wishlist_ID, Product_ID
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.12 Review Table

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text, Review_Date

Each review has a unique Review_ID that determines the customer, product, rating, review text, and review date.

Redundancy Analysis

Reviews are stored in a separate table instead of inside the Product or Customer tables. This allows multiple customers to review the same product without duplicating customer or product information. The use of foreign keys maintains normalization.

Data Anomalies

- Insert Anomaly: A review can be added only after both the customer and product exist.
- Update Anomaly: Editing a review or rating requires updating only one record.
- Delete Anomaly: Deleting a review does not remove the customer or product details.

Summary

Item	Description
Primary Key	Review_ID
Functional Dependency	Review_ID → Customer_ID, Product_ID, Rating, Review_Text, Review_Date
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.13 Product_Image Table

Functional Dependency

Image_ID → Product_ID, Image_URL, Image_Type

Each image is uniquely identified by Image_ID.

Redundancy Analysis

A product can have multiple images. Storing images in a separate table avoids adding multiple image columns to the Product table, making the design more flexible and normalized.

Data Anomalies

- Insert Anomaly: Images can be added only for existing products.
- Update Anomaly: Changing an image URL affects only one record.
- Delete Anomaly: Removing an image does not delete the product.

Summary

Item	Description
Primary Key	Image_ID
Functional Dependency	Image_ID $\rightarrow$ Product_ID, Image_URL, Image_Type
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.13 Notification Table

Functional Dependency

Notification_ID $\rightarrow$ Customer_ID, Message, Notification_Date, Read_Status

Each notification is uniquely identified by Notification_ID.

Redundancy Analysis

Notifications are stored separately from customer information so that a customer can receive multiple notifications without repeating customer details.

Data Anomalies

- Insert Anomaly: Notifications can be created only for registered customers.
- Update Anomaly: Updating the read status affects only one notification record.
- Delete Anomaly: Deleting a notification does not remove the customer account.

Summary

Item	Description
Primary Key	Notification_ID
Functional Dependency	Notification_ID $\rightarrow$ Customer_ID, Message, Notification_Date, Read_Status
Redundancy	Very Low

Item	Description
Anomalies	Minimized through normalization

4.3.14 AI_Recommendation Table

Functional Dependency

Recommendation_ID → Customer_ID, Product_ID, Recommendation_Score, Recommendation_Date

Each recommendation is uniquely identified by Recommendation_ID.

Redundancy Analysis

AI recommendations are stored separately instead of inside the Customer or Product tables. This allows multiple recommendations to be generated without duplicating customer or product data.

Data Anomalies

- Insert Anomaly: Recommendations can be created only after customer and product records exist.
- Update Anomaly: Updating the recommendation score affects only one record.
- Delete Anomaly: Removing a recommendation does not delete customer or product information.

Summary

Item	Description
Primary Key	Recommendation_ID
Functional Dependency	Recommendation_ID → Customer_ID, Product_ID, Recommendation_Score, Recommendation_Date
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.15 AI_Search_History Table

Functional Dependency

Search_ID → Customer_ID, Search_Query, Search_Date

Each search history record is uniquely identified by Search_ID.

Redundancy Analysis

Search history is maintained independently so that customers can perform multiple searches without affecting customer records.

Data Anomalies

- Insert Anomaly: Search history can be recorded only for existing customers.
- Update Anomaly: Updating a search record requires changing only one row.
- Delete Anomaly: Deleting search history does not affect customer details.

Summary

Item	Description
Primary Key	Search_ID
Functional Dependency	Search_ID → Customer_ID, Search_Query, Search_Date
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.16 Analytics Table

Functional Dependency

Analytics_ID → Product_ID, Total_Sales, Total_Revenue, Average_Rating, Last_Updated, Popularity_Score

Each analytics record is uniquely identified by Analytics_ID.

Redundancy Analysis

Analytics data is stored separately because it contains calculated business information rather than operational data. This keeps transactional tables simple and improves performance.

Data Anomalies

- Insert Anomaly: Analytics can be created only after product information exists.
- Update Anomaly: Updating sales statistics affects only one analytics record.
- Delete Anomaly: Deleting analytics does not remove product information.

Summary

Item	Description
Primary Key	Analytics_ID

Item	Description
Functional Dependency	Analytics_ID → Product_ID, Total_Sales, Total_Revenue, Average_Rating, Last_Updated, Popularity_Score
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.17 Order_Status_History Table

Functional Dependency

Status_ID → Order_ID, Status, Updated_Time

Each status history record stores one status update for an order.

Redundancy Analysis

Instead of replacing the current order status repeatedly, historical status records are stored separately. This preserves tracking history while avoiding unnecessary duplication.

Data Anomalies

- Insert Anomaly: Status history can be added only for existing orders.
- Update Anomaly: Correcting a status update affects only one history record.
- Delete Anomaly: Deleting one status entry does not remove the order.

Summary

Item	Description
Primary Key	Status_ID
Functional Dependency	Status_ID → Order_ID, Status, Updated_Time
Redundancy	Very Low
Anomalies	Minimized through normalization

4.3.18 Admin Table

Functional Dependency

Admin_ID → Admin_Name, Email, Password

Each administrator is uniquely identified by Admin_ID.

Redundancy Analysis

Administrative information is maintained separately from customer information because administrators have different responsibilities and permissions within the system.

Data Anomalies

- Insert Anomaly: A new administrator can be added independently.
- Update Anomaly: Updating administrator details affects only one record.
- Delete Anomaly: Removing an administrator does not affect customers or orders.

Summary

Item	Description
Primary Key	Admin_ID
Functional Dependency	Admin_ID → Admin_Name, Email, Password
Redundancy	Very Low
Anomalies	Minimized through normalization

4.4 Overall Database Analysis

The IKEA Furniture E-commerce database has been designed by separating different business functions into individual tables. Each table stores only information related to a single entity, while relationships are maintained using primary keys and foreign keys. This design significantly reduces duplicate data and improves the overall quality of the database.

Overall Redundancy Reduction

- Customer information is stored only once in the Customer table.
- Product information is maintained separately from categories, suppliers, and inventory.
- Order details, payment information, shipment tracking, and return requests are stored in independent tables.
- Many-to-many relationships are resolved using bridge tables such as Order_Item, Cart_Item, and Wishlist_Item.
- Foreign keys eliminate unnecessary duplication of data.

As a result, the database contains very little redundancy.

4.5 Overall Normalization

The database follows normalization principles by ensuring that:

- Each table represents a single entity.
- Every table has a primary key.
- Non-key attributes depend only on the primary key.
- Related information is connected through foreign keys rather than duplication.

This makes the database easier to maintain and extend.

Data Consistency

Since each piece of information is stored only once, updates are consistent throughout the database. For example:

- Updating a product price automatically affects future orders without changing customer records.
- Updating customer information requires modification in only one table.

This improves data accuracy.

Referential Integrity

Foreign keys ensure that relationships remain valid.

Examples include:

- Every order belongs to an existing customer.
- Every review belongs to an existing customer and product.
- Every shipment belongs to an existing order.
- Every inventory record references a valid warehouse and product.

This prevents invalid or orphan records.

Advantages of Foreign Keys

- Maintain relationships between tables.
- Prevent invalid data entry.
- Improve consistency.
- Reduce redundancy.
- Simplify SQL JOIN operations.
- Improve overall database reliability.

Scalability

The modular database design makes future expansion easy.

New features such as:

- Loyalty programs
- Gift cards
- Product recommendations
- AI modules
- Multiple payment gateways
- Seller marketplace

can be added without redesigning the existing database.

Therefore, the database is scalable and suitable for future growth.

The database supports all major business operations required by a modern furniture e-commerce platform, including:

- Customer account management
- Product catalog management
- Shopping cart
- Wishlist
- Order processing
- Online payments
- Shipment tracking
- Inventory management
- Product reviews
- Coupon management
- Return and refund processing
- AI-based recommendations
- Business analytics
- Notifications

These features make the database practical, organized, and suitable for a real-world furniture e-commerce application.

5. Conclusion

The analysis of the IKEA Furniture E-commerce database shows that the database has been designed using proper relational database principles. Functional dependencies clearly define how each primary key determines the remaining attributes in every table, ensuring that data is stored logically and consistently.

By separating customers, products, orders, payments, inventory, reviews, suppliers, and other business functions into individual tables, the database minimizes redundancy and avoids unnecessary duplication of information. Foreign keys establish relationships between tables while maintaining referential integrity and ensuring accurate data connections.

The design also reduces insert, update, and delete anomalies. New records can be inserted without unnecessary duplication, updates are performed in a single location, and deleting records does not unintentionally remove related information. This improves data consistency and reliability.

The normalized structure makes the database easier to maintain, supports efficient SQL queries, and allows future enhancements such as advanced AI features, recommendation systems, additional payment methods, and business analytics without major structural changes.

Overall, the database is efficient, scalable, reliable, and well-suited for a real-world furniture e-commerce platform. It provides a strong foundation for implementing database operations while following industry-standard database design practices.